

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Разработка примитивной системы защиты на уровне СУБД»

Выполнили:

Шибает Иван Дмитриевич, студент группы N3352

(подпись)

Малышев Григорий Игоревич, студент группы
N3352

(подпись)

Проверил:

Ярцева Наталия Андреевна

(подпись)

СОДЕРЖАНИЕ

Введение	3
1 Подготовка стенда	4
2 Мониторинг изменений	5
2.1 Таблица-журнал и триггерная функция	5
2.2 Демонстрация журналирования	8
3 Шифрование секретных данных	11
3.1 Таблица секретов и алгоритм	11
3.2 Демонстрация	13
4 Разграничение доступа	15
4.1 Классы потребителей и привилегии	15
4.2 Демонстрация разграничения доступа	19
Заключение	24

ВВЕДЕНИЕ

Цель работы — получение навыков создания примитивных систем мониторинга, разграничения доступа и шифрования средствами СУБД.

Задание включает три группы задач:

- мониторинг: создать отдельную таблицу-журнал и триггеры для каждой основной таблицы, которые при вставке, изменении и удалении записывают время, роль, изменённые кортежи, старые и новые значения; продемонстрировать работу журнала;
- шифрование: создать отдельную таблицу секретных данных и зашифровать её симметричным алгоритмом; ключ не хранится в системе и выводится из индивидуального пароля суперпользователя односторонней функцией; показать, что права администратора без пароля не позволяют прочитать данные;
- разграничение доступа: создать роли для классов потребителей информации, назначить им минимально необходимые привилегии на таблицы и представления, закрыть журнал от всех, кроме суперпользователя, и продемонстрировать доступ от имени каждой роли.

Работа выполнена на базе данных «workshop» мастерской реставрации музыкальных инструментов, созданной в лабораторной работе № 2 в PostgreSQL 14.21.

1 Подготовка стенда

Все действия воспроизводятся сценарием «check.sh» (команда «make check»). Он удаляет прежние базу данных и роли, заново строит базу данных сценарием лабораторной работы № 2, выполняет три скрипта этой работы, а затем подключается к серверу от имени созданных учётных записей и сохраняет выводы демонстраций в каталог «verification». Все листинги результатов в отчёте получены этим сценарием.

Скрипты выполняются в следующем порядке:

1. «sql/01_audit.sql» — таблица-журнал, триггерная функция и триггеры;
2. «sql/02_secrets.sql» — таблица секретов и её шифрование;
3. «sql/03_roles.sql» — роли, привилегии и политики строк.

Перед демонстрациями журнал очищается командой «TRUNCATE ... RESTART IDENTITY», чтобы в нём остались только операции демонстрационных сеансов. Пароли учётных записей и индивидуальный пароль передаются в скрипты переменными psql из переменных окружения; в репозитории заданы только значения по умолчанию для демонстрации.

2 Мониторинг изменений

2.1 Таблица-журнал и триггерная функция

Журнал «change_log» хранит для каждой изменённой строки: момент изменения, роль, под которой клиент вошёл в СУБД, адрес клиента, номер транзакции, тип операции (I, U, D), имя таблицы, значение первичного ключа и полные образы строки до и после изменения в формате «jsonb». Код приведён в листинге 1.

```
-- Журнал изменений основных таблиц.
BEGIN;

CREATE TABLE change_log (
    log_id      bigint
               GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    changed_at  timestampz NOT NULL DEFAULT clock_timestamp(),
    db_role     text NOT NULL,
    client      inet,
    tx_id       bigint NOT NULL,
    operation   char(1) NOT NULL
               CHECK (operation IN ('I', 'U', 'D')),
    table_name  text NOT NULL,
    row_key     jsonb NOT NULL,
    old_row     jsonb,
    new_row     jsonb
);

CREATE INDEX change_log_table_key_idx
ON change_log (table_name, row_key);
CREATE INDEX change_log_changed_at_idx
ON change_log (changed_at);

-- SECURITY DEFINER: запись в журнал выполняется от владельца,
-- поэтому прикладным ролям не нужны права на change_log.
-- session_user – роль, под которой вошёл клиент; current_user
-- внутри функции равен владельцу и для журнала не подходит.
CREATE FUNCTION log_change() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = public, pg_temp
AS $$
DECLARE
```

```

src    jsonb := to_jsonb(CASE WHEN TG_OP = 'DELETE'
                           THEN OLD ELSE NEW END);

k      jsonb := '{}';
col    text;
BEGIN
    -- Аргументы триггера – столбцы первичного ключа.
    FOREACH col IN ARRAY TG_ARGV LOOP
        k := k || jsonb_build_object(col, src -> col);
    END LOOP;

    INSERT INTO change_log (db_role, client, tx_id,
                           operation, table_name, row_key, old_row, new_row)
    VALUES (session_user, inet_client_addr(), txid_current(),
            left(TG_OP, 1), TG_TABLE_NAME, k,
            CASE WHEN TG_OP <> 'INSERT' THEN to_jsonb(OLD) END,
            CASE WHEN TG_OP <> 'DELETE' THEN to_jsonb(NEW) END);
    RETURN NULL;
END;
$$;

REVOKE ALL ON FUNCTION log_change() FROM PUBLIC;

DO $$
DECLARE
    t record;
BEGIN
    FOR t IN SELECT * FROM (VALUES
        ('customer', 'customer_id'),
        ('instrument_type', 'type_id'),
        ('instrument', 'instrument_id'),
        ('employee', 'employee_id'),
        ('restoration_order', 'order_id'),
        ('service', 'service_id'),
        ('order_work', 'order_id, line_no'),
        ('inspection', 'inspection_id'),
        ('payment', 'payment_id')
    ) AS v(tab, pk) LOOP
        EXECUTE format(
            'CREATE TRIGGER %I '
            'AFTER INSERT OR UPDATE OR DELETE ON %I '
            'FOR EACH ROW EXECUTE FUNCTION log_change(%s)',
            t.tab || '_log', t.tab,
            (SELECT string_agg(quote_literal(btrim(c)), ', ')

```

```

        FROM unnest(string_to_array(t.pk, ',')) AS c));
    END LOOP;
END $$;

-- Журнал доступен только суперпользователю.
REVOKE ALL ON change_log FROM PUBLIC;

COMMIT;

```

Листинг 1 — Журнал и триггеры («sql/01_audit.sql»)

Решения, принятые при проектировании:

- Функция «log_change()» объявлена как «SECURITY DEFINER» и выполняется с правами владельца. Поэтому прикладным ролям не выдаются права на журнал: они не могут ни прочитать, ни подделать записи прямой вставкой. Для такой функции явно задан «search_path», что исключает подмену объектов через временную схему.
- Внутри функции с «SECURITY DEFINER» переменная «current_user» равна владельцу функции. Поэтому в журнал записывается «session_user» — роль, под которой клиент прошёл аутентификацию.
- Время берётся функцией «clock_timestamp()», а не «now()», чтобы у нескольких изменений одной транзакции были фактические моменты выполнения.
- Одна функция обслуживает все таблицы: имена столбцов первичного ключа передаются аргументами триггера, а старое и новое значения строки сериализуются функцией «to_jsonb». Для составного ключа «order_work» передаются два столбца.
- Триггеры создаются циклом для всех девяти основных таблиц со срабатыванием «AFTER INSERT OR UPDATE OR DELETE FOR EACH ROW»: в журнал попадают только успешно выполненные изменения. Таблица секретов не журналируется: её содержимое и так зашифровано, а в журнал попадали бы копии шифртекста.

Созданные триггеры, прочитанные из каталога «pg_trigger», приведены в листинге 2.

trigger	after_events
customer_log	INSERT OR DELETE OR UPDATE
employee_log	INSERT OR DELETE OR UPDATE
inspection_log	INSERT OR DELETE OR UPDATE
instrument_log	INSERT OR DELETE OR UPDATE
instrument_type_log	INSERT OR DELETE OR UPDATE
order_work_log	INSERT OR DELETE OR UPDATE
payment_log	INSERT OR DELETE OR UPDATE
restoration_order_log	INSERT OR DELETE OR UPDATE
service_log	INSERT OR DELETE OR UPDATE
(9 rows)	

Листинг 2 — Триггеры журналирования

2.2 Демонстрация журналирования

В журнал попали операции трёх сеансов: приёмщика (листинг 15), реставратора (листинг 16) и суперпользователя, удалившего тестовую оплату (листинг 3). Для каждой записи известны время, роль, операция, таблица, транзакция, адрес клиента и ключ строки. Операции, отклонённые СУБД, в журнал не попали: они откатываются вместе с записью триггера.

```
DELETE FROM payment WHERE receipt_no = 'DEMO-009';
DELETE 1
SELECT log_id AS id,
       to_char(changed_at, 'HH24:MI:SS') AS changed_at,
       db_role, operation AS op, table_name
FROM change_log ORDER BY log_id;
```

id	changed_at	db_role	op	table_name
1	12:40:11	reception_orlova	I	customer
2	12:40:11	reception_orlova	I	instrument
3	12:40:11	reception_orlova	I	restoration_order
4	12:40:11	reception_orlova	I	payment
5	12:40:11	reception_orlova	U	customer
6	12:40:11	reception_orlova	U	restoration_order
7	12:40:11	restorer_smirnov	U	order_work
8	12:40:11	restorer_smirnov	I	inspection
9	12:40:11	postgres	D	payment
(9 rows)				

```

SELECT log_id AS id, tx_id, client, row_key
FROM change_log ORDER BY log_id;
  id | tx_id |  client  |          row_key
-----+-----+-----+-----
  1 | 1175 | 127.0.0.1 | {"customer_id": 9}
  2 | 1176 | 127.0.0.1 | {"instrument_id": 10}
  3 | 1177 | 127.0.0.1 | {"order_id": 10}
  4 | 1178 | 127.0.0.1 | {"payment_id": 10}
  5 | 1179 | 127.0.0.1 | {"customer_id": 9}
  6 | 1180 | 127.0.0.1 | {"order_id": 10}
  7 | 1181 | 127.0.0.1 | {"line_no": 1, "order_id": 1}
  8 | 1182 | 127.0.0.1 | {"inspection_id": 9}
  9 | 1183 | 127.0.0.1 | {"payment_id": 10}
(9 rows)

```

Листинг 3 — Удаление оплаты суперпользователем и содержимое журнала

Старые и новые значения хранятся полностью. Запрос из листинга 4 сравнивает образы строк и выводит только изменённые поля, а для удаления — всю удалённую строку. Результат приведён в листинге 5: видно изменение электронной почты заказчика и срока заказа приёмщиком, смену статуса работы реставратором и содержимое удалённой оплаты.

```

-- Поля, изменённые операциями UPDATE: было и стало.
SELECT l.log_id AS id, e.key AS field,
       l.old_row -> e.key AS old_value, e.value AS new_value
FROM change_log l,
     jsonb_each(l.new_row) AS e
WHERE l.operation = 'U'
     AND l.old_row -> e.key IS DISTINCT FROM e.value
ORDER BY l.log_id, e.key;
-- Удалённая строка сохраняется целиком в old_row.
SELECT log_id AS id, jsonb_pretty(old_row) AS deleted_row
FROM change_log WHERE operation = 'D';

```

Листинг 4 — Запрос изменённых значений («demo/log_diff.sql»)

```

  id | field  | old_value | new_value
-----+-----+-----+-----
  5 | email  | null      | "belova@example.test"
  6 | due_on | "2026-10-03" | "2026-10-01"
  7 | status | "in_progress" | "done"

```

(3 rows)

id	deleted_row
9	{
	"method": "card",
	"paid_on": "2026-09-19",
	"order_id": 10,
	"payment_id": 10,
	"receipt_no": "DEMO-009",
	"amount_kopecks": 100000
	}

(1 row)

Листинг 5 — Старые и новые значения из журнала

3 Шифрование секретных данных

3.1 Таблица секретов и алгоритм

Таблица «access_secret» отделена от основных сущностей и хранит секреты классов пользователей: токены платёжного терминала и SMS-шлюза для приёмщиков, учётные данные каталога поставщика и станка для реставраторов. Название и значение секрета хранятся только в зашифрованном виде; в открытом виде оставлен лишь класс потребителя, по которому выбирается нужная запись.

Шифрование выполняется расширением pgcrypto (листинг 6):

1. ключ вычисляется как шестнадцатеричная запись SHA-256 от индивидуального пароля суперпользователя, не связанного с паролем входа в СУБД; хэш-функция односторонняя, и по ключу пароль не восстанавливается;
2. функция «pgp_sym_encrypt» с параметром «cipher-algo=aes256» шифрует значения алгоритмом AES-256 в формате OpenPGP. Для каждого значения формируются случайные соль и вектор инициализации, поэтому одинаковые открытые тексты дают разные шифртексты. Формат содержит код обнаружения изменений: расшифровка неверным ключом или изменённого шифртекста завершается ошибкой, а не выдаёт мусор.

```
-- Таблица секретов классов пользователей, шифрование AES-256.  
-- Запуск: psql -v master_password=... -f 02_secrets.sql  
-- Ключ = SHA-256 от индивидуального пароля; в БД ни пароль,  
-- ни ключ не сохраняются.
```

```
BEGIN;
```

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

```
CREATE TABLE access_secret (  
    secret_id      integer  
        GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    consumer_class text NOT NULL,  
    secret_name    bytea NOT NULL,  
    secret_value   bytea NOT NULL  
);
```

```

REVOKE ALL ON access_secret FROM PUBLIC;

-- Ключ вычисляется только на время выполнения этого скрипта.
SELECT encode(digest(:'master_password', 'sha256'), 'hex')
      AS k \gset

INSERT INTO access_secret (consumer_class, secret_name,
                          secret_value)

SELECT class,
       pgp_sym_encrypt(name, :'k', 'cipher-algo=aes256'),
       pgp_sym_encrypt(value, :'k', 'cipher-algo=aes256')
FROM (VALUES
      ('reception', 'Токен платёжного терминала',
       'term_7Qm2-demo-4hXv9Lp'),
      ('reception', 'Ключ SMS-шлюза уведомлений',
       'sms_K3fd8-demo-WqlzR0'),
      ('restorer', 'Доступ к каталогу поставщика',
       'cat_P9sd1-demo-Yt6uB2'),
      ('restorer', 'Учётная запись станка ЧПУ',
       'cnc_H5gn4-demo-Ze7kM3')
) AS v(class, name, value);

\unset k
COMMIT;

```

Листинг 6 — Таблица секретов («sql/02_secrets.sql»)

Пароль передаётся скрипту переменной psql «master_password». Ключ записывается в клиентскую переменную командой «\gset», используется в одном операторе «INSERT» и удаляется командой «\unset». Ни пароль, ни ключ не сохраняются в таблицах, функциях или журнале. Проверка в листинге 7 ищет пароль и ключ в исходных текстах всех функций, в таблице секретов и в журнале.

строк с ключом или паролем в БД: 0

Листинг 7 — Поиск пароля и ключа в базе данных

Доступ к «access_secret» отозван у PUBLIC и не выдаётся ни одной прикладной роли (листинги 15 и 16). Таблицу может прочитать только суперпользователь, но и он получает лишь шифртекст.

3.2 Демонстрация

Запросы демонстрации приведены в листинге 8, результат — в листинге 9. Суперпользователь видит только шифртекст: пакеты OpenPGP начинаются с одинакового заголовка «c30d0409», но отличаются солью. Попытка расшифровки ключом, полученным из другого пароля, завершается ошибкой «Wrong key or corrupt data». После ввода индивидуального пароля все четыре секрета расшифрованы.

```
-- Ключ = SHA-256 от индивидуального пароля (переменная psql).
SELECT encode(digest(:'master_password', 'sha256'), 'hex')
       AS key \gset
-- Суперпользователь видит только шифртекст.
SELECT secret_id AS id, consumer_class,
       substr(encode(secret_value, 'hex'), 1, 32) || '...'
       AS secret_value_hex
FROM access_secret ORDER BY secret_id;
-- Расшифровка с неверным паролем.
SELECT pgp_sym_decrypt(secret_value,
       encode(digest(:'wrong_password', 'sha256'), 'hex'))
FROM access_secret WHERE secret_id = 1;
-- Расшифровка с индивидуальным паролем.
SELECT secret_id AS id,
       pgp_sym_decrypt(secret_name, :'key') AS name,
       pgp_sym_decrypt(secret_value, :'key') AS value
FROM access_secret ORDER BY secret_id;
```

Листинг 8 — Демонстрация шифрования («demo/secrets.sql»)

id	consumer_class	secret_value_hex
1	reception	c30d04090302e7c0065f5422619672d2...
2	reception	c30d04090302b385ddf3f9f10d1e65d2...
3	restorer	c30d04090302309989b29cdf996161d2...
4	restorer	c30d04090302ec4ae48f91e1f37d6fd2...

(4 rows)

ERROR: Wrong key or corrupt data

id	name	value
1	Токен платёжного терминала	term_7Qm2-demo-4hXv9Lp
2	Ключ SMS-шлюза уведомлений	sms_K3fd8-demo-Wq1zR0
3	Доступ к каталогу поставщика	cat_P9sd1-demo-Yt6uB2

Листинг 9 — Шифртекст, неверный и верный пароль

Шифрование на стороне сервера имеет ограничение: в момент расшифровки ключ передаётся серверу в тексте запроса. Суперпользователь, заранее включивший протоколирование всех операторов («log_statement = all») или наблюдающий за «pg_stat_activity», может его перехватить. На стенде протоколирование операторов выключено. Для полной защиты от администратора расшифровку следует переносить в клиентское приложение, чтобы ключ не покидал его.

4 Разграничение доступа

4.1 Классы потребителей и привилегии

В лабораторной работе № 1 выделены два класса потребителей информации: приёмщик и реставратор. Для каждого класса создана групповая роль без права входа, а для конкретных сотрудников — индивидуальные учётные записи с правом входа, наследующие привилегии групповой роли. Суперпользователь «postgres» отдельной ролью класса не считается.

Привилегии определены по задачам классов из первой работы и принципу минимальных привилегий. Итоговая матрица приведена в таблице 1. Обозначения: S — чтение, I — вставка, U — изменение, D — удаление; звёздочка — привилегия только на перечисленные столбцы.

Таблица 1 — Привилегии групповых ролей

Объект	Приёмщик	Реставратор
customer	S, I, U	—
instrument	S, I, U	S* без customer_id
instrument_type	S	S
restoration_order	S, I, U	S* без closed_on
payment	S, I, U	—
service	S	S* без base_price_kopecks
employee	—	S* без work_email, db_login
order_work	—	S* без цены; U* только status, только свои строки
inspection	—	S; I* только от своего имени
v_reception_orders, v_reception_balances	S	—
v_restorer_tasks, v_restorer inspections	—	S
change_log, access_secret	—	—

Приёмщик регистрирует заказчиков, инструменты, заказы и оплаты, но не может удалять записи: ошибочные данные исправляются, а не стираются, и история остаётся в журнале. Данные сотрудников, работы и представления реставратора ему не нужны.

Реставратор не получает контактов заказчика, связи инструмента с заказчиком и денежных сумм — ни в представлениях, ни в базовых таблицах, где права выданы на отдельные столбцы. Столбцовых привилегий недостаточно, чтобы реставратор менял только свои работы. Поэтому

для «order_work» и «inspection» включена защита на уровне строк: изменять статус можно только в строках своей работы, регистрировать осмотр — только от своего имени. Сотрудник определяется функцией «current_employee_id()» по новому столбцу «employee.db_login», который связывает учётную запись СУБД с записью сотрудника.

Дополнительно у PUBLIC отозваны права подключения к базе данных и создания объектов в схеме «public», выдаваемые PostgreSQL по умолчанию. Подключиться могут только члены групповых ролей и суперпользователь. Учётные записи входят по паролю; сервер хранит пароли как хэш SCRAM-SHA-256 и не принимает их в открытом виде.

Код создания ролей и назначения привилегий приведён в листинге 10.

```
-- Разграничение доступа: групповые роли классов потребителей
-- из ЛР1 и индивидуальные учётные записи.
-- Пароли передаются переменными psql: reception_password,
-- restorer_password; сервер хранит только хэш SCRAM-SHA-256.
BEGIN;

-- 1. Убрать права, выдаваемые PUBLIC по умолчанию.
REVOKE CONNECT, TEMPORARY ON DATABASE workshop FROM PUBLIC;
REVOKE CREATE ON SCHEMA public FROM PUBLIC;

-- 2. Групповые роли (без входа).
CREATE ROLE reception_role NOLOGIN NOSUPERUSER NOCREATEDB
    NOCREATEROLE NOINHERIT;
CREATE ROLE restorer_role NOLOGIN NOSUPERUSER NOCREATEDB
    NOCREATEROLE NOINHERIT;
GRANT CONNECT ON DATABASE workshop
    TO reception_role, restorer_role;
GRANT USAGE ON SCHEMA public TO reception_role, restorer_role;

-- 3. Приёмщик: заказчики, инструменты, заказы, оплаты.
-- Удаление не разрешено: ошибочные записи исправляются.
GRANT SELECT, INSERT, UPDATE
    ON customer, instrument, restoration_order, payment
    TO reception_role;
GRANT SELECT ON instrument_type, service TO reception_role;
GRANT SELECT ON v_reception_orders, v_reception_balances
    TO reception_role;
```

```

-- 4. Реставратор: без контактов заказчика и денежных сумм.
GRANT SELECT ON v_restorer_tasks, v_restorer_inspections,
    instrument_type TO restorer_role;
GRANT SELECT (service_id, name, description)
    ON service TO restorer_role;
GRANT SELECT (employee_id, full_name, specialization)
    ON employee TO restorer_role;
GRANT SELECT (instrument_id, type_id, inventory_no, name,
    maker, serial_no)
    ON instrument TO restorer_role;
GRANT SELECT (order_id, instrument_id, accepted_on, due_on,
    status, complaint)
    ON restoration_order TO restorer_role;
GRANT SELECT (order_id, line_no, service_id, employee_id,
    status),
    UPDATE (status)
    ON order_work TO restorer_role;
GRANT SELECT, INSERT (order_id, employee_id, inspected_on,
    conclusion)
    ON inspection TO restorer_role;

-- 5. Сопоставление учётной записи и сотрудника. Внутри
-- SECURITY DEFINER current_user равен владельцу функции,
-- поэтому используется session_user – роль входа.
ALTER TABLE employee ADD COLUMN db_login text UNIQUE;

CREATE FUNCTION current_employee_id() RETURNS integer
LANGUAGE sql STABLE SECURITY DEFINER
SET search_path = public, pg_temp
AS $$ SELECT employee_id FROM employee
    WHERE db_login = session_user $$;
REVOKE ALL ON FUNCTION current_employee_id() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION current_employee_id()
    TO restorer_role;

-- 6. Реставратор меняет статус только своих работ и
-- регистрирует осмотры только от своего имени.
ALTER TABLE order_work ENABLE ROW LEVEL SECURITY;
CREATE POLICY work_read ON order_work FOR SELECT
    TO restorer_role USING (true);
CREATE POLICY work_own_update ON order_work FOR UPDATE
    TO restorer_role
    USING (employee_id = current_employee_id())

```

```

    WITH CHECK (employee_id = current_employee_id());

ALTER TABLE inspection ENABLE ROW LEVEL SECURITY;
CREATE POLICY inspection_read ON inspection FOR SELECT
    TO restorer_role USING (true);
CREATE POLICY inspection_own_insert ON inspection FOR INSERT
    TO restorer_role
    WITH CHECK (employee_id = current_employee_id());

-- 7. Индивидуальные учётные записи.
CREATE ROLE reception_orlova LOGIN
    PASSWORD :'reception_password'
    INHERIT IN ROLE reception_role;
CREATE ROLE restorer_smirnov LOGIN
    PASSWORD :'restorer_password'
    INHERIT IN ROLE restorer_role;
CREATE ROLE restorer_volkova LOGIN
    PASSWORD :'restorer_password'
    INHERIT IN ROLE restorer_role;

UPDATE employee SET db_login = 'restorer_smirnov'
WHERE employee_id = 1;
UPDATE employee SET db_login = 'restorer_volkova'
WHERE employee_id = 2;

COMMIT;

```

Листинг 10 — Роли и привилегии («sql/03_roles.sql»)

Созданные роли и членство в групповых ролях показаны в листинге 11.

rolname	login	super	member_of
postgres	t	t	
reception_orlova	t	f	reception_role
reception_role	f	f	
restorer_role	f	f	
restorer_smirnov	t	f	restorer_role
restorer_volkova	t	f	restorer_role

(6 rows)

Листинг 11 — Роли кластера

4.2 Демонстрация разграничения доступа

Демонстрация выполнена настоящим входом в СУБД по паролю под учётными записями «reception_orlova» и «restorer_smirnov», а не переключением «SET ROLE» из сеанса суперпользователя. Так проверяются и аутентификация, и то, какая роль попадает в журнал.

Для каждой учётной записи запрос из листинга 12 перечисляет отношения схемы «public», к которым у неё есть хотя бы одна привилегия. Результаты приведены в листингах 13 и 14 и совпадают с таблицей 1. Журнал и таблица секретов не видны ни одной из ролей.

```
-- Доступные текущей роли отношения схемы public.
-- S* – чтение только части столбцов.
SELECT c.relname AS relation,
       CASE WHEN has_table_privilege(c.oid, 'SELECT') THEN 'S'
            WHEN has_any_column_privilege(c.oid, 'SELECT')
            THEN 'S*'
            ELSE '' END AS sel,
       CASE WHEN has_table_privilege(c.oid, 'INSERT') THEN 'I'
            WHEN has_any_column_privilege(c.oid, 'INSERT')
            THEN 'I*'
            ELSE '' END AS ins,
       CASE WHEN has_table_privilege(c.oid, 'UPDATE') THEN 'U'
            WHEN has_any_column_privilege(c.oid, 'UPDATE')
            THEN 'U*'
            ELSE '' END AS upd,
       CASE WHEN has_table_privilege(c.oid, 'DELETE')
            THEN 'D' ELSE '' END AS del
FROM pg_class c
WHERE c.relnamespace = 'public'::regnamespace
      AND c.relkind IN ('r', 'v')
      AND has_any_column_privilege(c.oid,
                                   'SELECT, INSERT, UPDATE')
ORDER BY c.relkind, c.relname;
```

Листинг 12 — Запрос доступных отношений («demo/access_matrix.sql»)

relation	sel	ins	upd	del
customer	S	I	U	
instrument	S	I	U	
instrument_type	S			

payment	S	I	U	
restoration_order	S	I	U	
service	S			
v_reception_balances	S			
v_reception_orders	S			

(8 rows)

Листинг 13 — Отношения, доступные приёмщику «reception_orlova»

relation	sel	ins	upd	del
-----+-----+-----+-----				
employee	S*			
inspection	S	I*		
instrument	S*			
instrument_type	S			
order_work	S*		U*	
restoration_order	S*			
service	S*			
v_restorer inspections	S			
v_restorer_tasks	S			

(9 rows)

Листинг 14 — Отношения, доступные реставратору «restorer_smirnov»

Сеанс приёмщика приведён в листинге 15. Приёмщик принял новый заказ: зарегистрировал заказчика, инструмент, заказ и оплату, исправил почту и срок и увидел заказ в своём представлении. Попытки прочитать данные сотрудников, работы, представление реставратора, журнал, секреты и удалить оплату отклонены с ошибкой «permission denied».

```
SELECT current_user, session_user;
current_user | session_user
-----+-----
reception_orlova | reception_orlova
(1 row)
```

```
INSERT INTO customer (full_name, phone)
VALUES ('Белова Ксения Олеговна', '+7 900 000-00-09')
RETURNING customer_id AS cust
INSERT 0 1
INSERT INTO instrument (customer_id, type_id,
inventory_no, name)
```

```

VALUES (9, 2, 'INS-009', 'Гитара концертная')
RETURNING instrument_id AS inst
INSERT 0 1
INSERT INTO restoration_order (instrument_id, accepted_on,
                                due_on, status, complaint)
VALUES (10, '2026-09-19', '2026-10-03', 'accepted',
        'Отклеилась подставка')
RETURNING order_id AS ord
INSERT 0 1
INSERT INTO payment (order_id, paid_on, amount_kopecks,
                     method, receipt_no)
VALUES (10, '2026-09-19', 100000, 'card', 'DEMO-009');
INSERT 0 1
UPDATE customer SET email = 'belova@example.test'
WHERE customer_id = 9;
UPDATE 1
UPDATE restoration_order SET due_on = '2026-10-01'
WHERE order_id = 10;
UPDATE 1
SELECT order_id, customer_name, due_on, status
FROM v_reception_orders WHERE order_id = 10;
  order_id |      customer_name      |   due_on   | status
-----+-----+-----+-----
          10 | Белова Ксения Олеговна | 2026-10-01 | accepted
(1 row)

SELECT full_name, work_email FROM employee;
ERROR:  permission denied for table employee
SELECT * FROM order_work;
ERROR:  permission denied for table order_work
SELECT * FROM v_restorer_tasks;
ERROR:  permission denied for view v_restorer_tasks
SELECT count(*) FROM change_log;
ERROR:  permission denied for table change_log
SELECT count(*) FROM access_secret;
ERROR:  permission denied for table access_secret
DELETE FROM payment WHERE order_id = 10;
ERROR:  permission denied for table payment

```

Листинг 15 — Сеанс приёмщика

Сеанс реставратора приведён в листинге 16. Реставратор определён как сотрудник № 1, видит свою очередь, завершил свою работу и

зарегистрировал осмотр. Изменение чужой работы не затронуло ни одной строки («UPDATE 0»), а осмотр от имени другого сотрудника отклонён политикой строк. Чтение контактов, цены работы, связи инструмента с заказчиком, оплат, расчётов, журнала, секретов и изменение цены отклонены.

```
SELECT current_user, current_employee_id();
      current_user | current_employee_id
-----+-----
restorer_smirnov | 1
(1 row)
```

```
SELECT order_id, line_no, service_name, work_status
FROM v_restorer_tasks WHERE employee_id = 1;
      order_id | line_no |      service_name      | work_status
-----+-----+-----+-----
          1 | 1 | Склейка трещины деки | in_progress
(1 row)
```

```
UPDATE order_work SET status = 'done'
WHERE order_id = 1 AND line_no = 1;
UPDATE 1
INSERT INTO inspection (order_id, employee_id, inspected_on,
                        conclusion)
VALUES (1, 1, '2026-09-19', 'Склейка завершена');
INSERT 0 1
UPDATE order_work SET status = 'done'
WHERE order_id = 2 AND line_no = 1;
UPDATE 0
INSERT INTO inspection (order_id, employee_id, inspected_on,
                        conclusion)
VALUES (2, 2, '2026-09-19', 'Попытка подписи чужим именем');
ERROR: new row violates row-level security policy for table
"inspection"
SELECT full_name, phone FROM customer;
ERROR: permission denied for table customer
SELECT agreed_price_kopecks FROM order_work;
ERROR: permission denied for table order_work
SELECT customer_id FROM instrument;
ERROR: permission denied for table instrument
SELECT * FROM payment;
ERROR: permission denied for table payment
```

```
SELECT * FROM v_reception_balances;
ERROR: permission denied for view v_reception_balances
SELECT count(*) FROM change_log;
ERROR: permission denied for table change_log
SELECT count(*) FROM access_secret;
ERROR: permission denied for table access_secret
UPDATE order_work SET agreed_price_kopecks = 0
WHERE order_id = 1 AND line_no = 1;
ERROR: permission denied for table order_work
```

Листинг 16 — Сеанс реставратора

Попытка входа с неверным паролем отклоняется на этапе аутентификации (листинг 17).

```
error: connection to server at "127.0.0.1", port 55432
failed: FATAL: password authentication failed for user
"restorer_smirnov"
```

Листинг 17 — Вход с неверным паролем

ЗАКЛЮЧЕНИЕ

Средствами PostgreSQL построена примитивная система защиты базы данных мастерской.

Мониторинг реализован таблицей-журналом и девятью триггерами. Журнал фиксирует время, роль входа, адрес клиента, транзакцию, операцию, таблицу, ключ строки, старые и новые значения. Прикладные роли его не видят и не могут подделать, так как запись выполняет функция с правами владельца.

Секреты классов пользователей хранятся в отдельной таблице, зашифрованной AES-256 ключом, который выводится из индивидуального пароля через SHA-256 и в базе данных не сохраняется. Показано, что суперпользователь без пароля получает только шифртекст, а неверный пароль приводит к ошибке расшифровки. Ограничение серверной расшифровки — передача ключа в тексте запроса — описано в отчёте.

Для приёмщика и реставратора созданы групповые роли и индивидуальные учётные записи с парольной аутентификацией SCRAM-SHA-256. Привилегии выданы по принципу минимальных привилегий, в том числе на уровне столбцов, а изменение работ и осмотров ограничено собственными строками сотрудника. Демонстрация от имени каждой учётной записи подтвердила, что разрешённые операции выполняются, а остальные отклоняются. Роли и права используются приложением в лабораторной работе № 4.